

Rendering a collection, modules ;

Rendering Collection

- Traditional Way :-

```
const App = ({notes}) => {  
  
  return (  
    <div>  
      <h1>Notes</h1>  
      <ul>  
        <li>{notes[0].content}</li>  
        <li>{notes[1].content}</li>  
        <li>{notes[2].content}</li>  
      </ul>  
    </div>  
  )  
};
```

- React Way :-

```
const App = ({ notes }) => {  
  
  return (  
    <div>  
      <h1>Notes</h1>  
      <ul>  
        {notes.map(note => <li>{note.content}</li>)}  
      </ul>  
    </div>  
  )  
};
```

Key-attribute

- Even though the application seems to be working, there is a nasty warning in the console:
- Each child in a list should have a unique "key" prop.
- in the error message suggests; the list items, i.e. the elements generated by the map method, must each have a unique key value: an attribute called key.

```
const App = ({ notes }) => {
```

```
return (  
  <div>  
    <h1>Notes</h1>  
    <ul>  
      {notes.map(note => <li key={note.id}>{note.content}</li>)}  
    </ul>  
  </div>  
)  
};
```

- And the error message disappears.
- React uses the **key attributes of objects in an array** to determine how to update the view generated by a component when the component is re-rendered.

Map

```
const result = notes.map(note => note.id)  
console.log(result)
```

- map always creates a new array, the elements of which have been created from the elements of the original array by mapping: using the function given as a parameter to the map method.

```
note => note.id
```

- Which is an arrow function written in compact form. The full form would be:

```
(note) => {  
  return note.id  
}
```

- The function gets a note object as a parameter and returns the value of its id field.

```
const result = notes.map(note => note.content)
```

- just we need used this snippt into react from

```
notes.map(note =>  
  <li key={note.id}>  
    {note.content}
```

```

    </li>
  )

```

- s used to create view elements, the value of the variable must be rendered inside curly braces.
- {} - curly braces is main factor of this section if you did not use wisely you will get in problem.

Anti-pattern: Array Indexes as Keys

- We could have made the error message on our console disappear by using the array indexes as keys.
- The indexes can be retrieved by passing a second parameter to the callback function of the map method:

```
const result = arr.map((ite, i) =>...);
```

- When called like this, `i` is assigned the value of the **index of the position** in the array where the note resides.

```

<ul>
  {notes.map((note, i) =>
    <li key={i}>
      {note.content}
    </li>
  )}
</ul>

```

- This is, however, **not recommended** and can create undesired problems even if it seems to be working just fine.

Refactoring Modules

- We'll separate displaying a single note into its own component Note:

```

const Note = ({ note }) => {
  return (
    <li>{note.content}</li>
  )
}

const App = ({ notes }) => {
  return (
    <div>
      <h1>Notes</h1>
      <ul>

```

```

    {notes.map(note =>
      <Note key={note.id} note={note} />
    )}
  </ul>
</div>
)
}

```

- The key attribute must ALWAYS be placed on the exact item that is being returned directly inside the `.map()` loop.
- Inside your component, your `.map()` loop is running like a factory conveyor belt. Every time the loop spins, it spits out a new box.

```

{notes.map(note =>
  <Note key={note.id} note={note} />
  // The factory stamps the ID on the OUTSIDE of the box here
)};

```

- React needs to be able to look at that conveyor belt and instantly identify every box. That is why the key MUST be stamped on the component itself.
- If you put the key on the
- inside the Note component; You are essentially hiding the ID card inside the box.

Refactoring Components

- A whole React application can be written in a single file. Although that is, of course, not very practical. Common practice is to declare each component in its own file as an ES6-module.
- In smaller applications, components are usually placed in a directory called components within the src directory. The convention is to name the file after the component. A possible directory structure for a project containing multiple components could be as follows:
- Do not forget to even we are separating Components into separate rule will be same for naming file convention first letter will always Capital like this `Note.jsx` , `Footer.jsx` ;

```

src/
├── main.jsx
├── App.jsx
└── components/           # Folder for reusable components
    ├── Footer.jsx       # File named after the component
    ├── Note.jsx
    └── Notification.jsx

```

- the main app logic and state stays trapped inside `const App = () => {}`. But each child component gets to hold its own tiny, specific logic too!
- `// Inside Note.jsx`

```
const Note = ({ note, onClickFn }) => {  
  
  // You can put tiny, specific logic just for the Note right here!  
  const capitalizedText = note.content.toUpperCase();  
  
  return (  
    <li onClick={onClickFn}>  
      {capitalizedText}  
    </li>  
  )  
}
```

Part2_a and Exercises 2.1 to 2.5 Veridict

- touches on the absolute core engine of React: The Virtual DOM.

Why is it mandatory?

- Imagine you have a list of 5 input boxes on the screen, and you type your name into the 3rd box. Then, you press a button that deletes the 1st box.
- Without keys, React gets confused. It looks at the new array and says: "Wait, there are only 4 boxes now. Let me just destroy all 5 boxes and redraw 4 new ones." If it does that, it deletes the name you just typed.
- With keys, React says: "Ah, Box ID #1 was deleted. But Box ID #2, #3, #4, and #5 are still here. I will just delete Box #1 and leave the others alone." Your text stays perfectly safe. Keys prevent massive UI bugs!

-
1. The Key Rule: The key goes on the absolute outermost box on the conveyor belt.
 2. The Curly Brace Rule: If you open an arrow function with `{}`, Return is God. If you forget it, you get the Invisible Man
- If you want to be a rebel and *skip the return keyword*, you have to use parentheses `()` instead of curly braces, which tells JavaScript to do an "implicit return". But honestly, just typing return is safer for now!